

Improving Step Accuracy in Specification-Driven Development



Agile Epic decomposition and test-driven development as an accuracy method for LLM builds

Ed Barlow Web Cloud Studio July 2026

Improving Step Accuracy in Specification-Driven Development

Ed Barlow — Web Cloud Studio

Abstract

Specification-driven development (SDD) builds software from a written specification by executing a sequence of LLM build steps. Sequential builds fail multiplicatively: a build of n steps, each correct with probability p , is correct with probability p^n . This paper describes a method that improves p and stops the decay. Agile Epic decomposition reduces the specification to small work items, each with acceptance criteria written before implementation. Test-driven development supplies the verification discipline: the acceptance criteria are executed after each step as deterministic checks, outside the model, so completion is measured rather than self-reported. Verified steps gate the steps that depend on them, so an error cannot propagate past a checked boundary. Two effects follow. Build correctness becomes a per-step property rather than a chain property, and the model capability required per step falls, because retention, error detection, and recovery move from the model into the process.

Keywords: specification-driven development, LLM code generation, test-driven development, Agile decomposition, acceptance criteria, verification, step accuracy

1. The Accuracy Problem

An LLM build is a chain of dependent steps. If each step is correct with independent probability p , the chain is correct with probability p^n . At $p = 0.95$ and $n = 40$, the final artifact is correct less than 13% of the time. This arithmetic, not model quality, explains why long builds drift and why re-running a build from the same specification rarely reproduces the same working software.

The arithmetic exposes two structural defects in current practice:

1. **Steps are too large to verify.** A step that implements many behaviors at once has no checkable success condition.
2. **Verification is self-reported.** When the model that did the work also judges the work, error detection fails exactly when it is needed.

Both defects were solved for human teams decades ago. Agile decomposes work into stories with acceptance criteria agreed before implementation [2]. Test-driven development writes the test before the code, so "done" is an executable fact rather than the author's claim [3]. This paper states the transfer of both methods to SDD as an engineering procedure.

2. Agile Epic Decomposition

The specification is treated as an Epic and decomposed by the established Agile method: Epic → features → stories.

A story is admissible when it satisfies three conditions:

Condition	Test
Bounded	Its required context — the specification sections it implements plus the interfaces it consumes — fits a declared token budget
Checkable	Its acceptance criteria can be written as executable assertions before implementation
Ordered	Its dependencies on other stories are explicit

A story that fails any condition is decomposed further. A story that cannot be given honest acceptance criteria is, by that fact, not yet a story.

Two supporting practices complete the decomposition:

Spikes. A question the specification cannot answer — a library choice, an unproven integration — becomes a spike: a work item whose product is a recorded answer, not code. The answer is written down once and supplied as context to every dependent story. Research is never re-performed inside a build step.

Refinement. Ambiguity discovered during decomposition is returned to the product owner as an explicit question and resolved before planning completes. Every ambiguity is settled by a human decision before it can become code. This is the standard Agile refinement loop; the only change is that the questions come from an LLM instead of a development team.

The output is a plan in which every work item carries its own verification contract and names its dependencies. Decomposition quality is measurable: the fraction of stories that pass their acceptance on first build.

3. Using Test-Driven Development to Improve Step Quality

TDD applies to SDD at story granularity, with one inversion and two rules.

The inversion: acceptance precedes build. Acceptance criteria are authored during planning, attached to the story, and included in the build prompt as the step's success condition. The build's task is to make declared assertions pass — not to interpret an open-ended instruction. Small, fully specified, criterion-anchored tasks are the regime in which model output is most reliable; the criteria are what create that regime.

Rule 1: verification is external and deterministic. After each step, the declared criteria run as ordinary test invocations — outside the model, consuming no context. The model is never asked whether it finished. A model asked "did you finish?" gives a sincere, unreliable answer; this is the same failure TDD was designed to remove from human self-assessment, and it is removed the same way: the test decides.

Rule 2: criteria are human-owned and monotonic. The build may add finer-grained tests. It may never remove or weaken a declared assertion. Alongside positive criteria, the specification carries permanent negative assertions — behaviors the software must never exhibit — which guard against model hallucination rather than specification omission and persist across rebuilds.

Criteria that cannot be honestly automated — visual quality, workflow judgment — are declared as human review items rather than skipped or dishonestly mechanized. The boundary between automated and human acceptance is explicit in the plan.

4. Error Containment

The decomposition graph and the verification discipline compose into a containment property:

- A step may run only when every step it depends on has passed its acceptance.
- A step that fails acceptance blocks its dependents.

An error is therefore detected at the step that created it, and the repair is local: revise that story's instructions or criteria and rerun one step. Without the gate, the same defect surfaces many steps later, embedded in dependent code, and the repair is archaeological.

The effect on the arithmetic of §1 is direct. Verification at every graph edge means no chain of unverified steps ever exceeds length one. Build correctness degrades from p^n to p per step, enforced n times — and §2 and §3 exist to raise that per-step p : bounded context, explicit criteria, resolved ambiguity.

The guarantee is scoped precisely: passing acceptance proves the software satisfies its *declared* criteria. Criteria quality remains a human responsibility, and the method places it where Agile always placed it — with the product owner, at refinement time.

5. Side Effect: Reduced Model Requirements

Long unverified builds require frontier models because the model must retain constraints across a large context, notice its own errors, and recover without external signal. The method externalizes each demand:

Demand on the model	Replaced by
Context retention	Small, bounded steps with complete declared context
Error detection	External deterministic acceptance checks
Recovery	Reopen-and-rerun of a single failed story

What remains is the task on which mid-tier and frontier models perform comparably: implement one bounded story against explicit criteria. Verification adds no model cost — it is ordinary test execution. Teams can reserve the strongest models for decomposition and planning, where judgment density is highest, and delegate story implementation to smaller models without weakening the correctness property, because that property never depended on the model's self-assessment. Verification effort, not model scale, carries the quality.

6. Related Work and Implementation

Spec Kit [4] established the specification-plus-task-list interface to coding agents; the method described here adds pre-declared acceptance, dependency gating, and external verification to that task model. Contemporary agent frameworks commonly evaluate completion by model self-report or by an LLM judge; the position of this paper is that deterministic, non-agentic verification is the only evaluation that composes across a long build. The author is not aware of another published SDD method combining pre-declared, human-owned acceptance criteria with dependency-gated deterministic verification.

Drydock [1] is the reference implementation of this method; a companion paper [5] describes the delivery-cost optimizations that the same decomposition enables.

7. Conclusion

Step accuracy in specification-driven development improves by process, not by model scale. Decompose the Epic until every story is bounded, checkable, and ordered. Write the acceptance criteria before the build and put them in the prompt. Verify every step externally and deterministically, and let verification gate the dependency graph. Errors stop propagating, correctness stops decaying with build length, and the model required for each step shrinks to fit the step.

References

- [1] E. Barlow. *Drydock Specification: Agile Specification-Driven Design — The SAIL Methodology for Governed Software Delivery*. Web Cloud Studio, 2026. <https://github.com/webcloudstudio/Drydock>
- [2] K. Beck et al. *Manifesto for Agile Software Development*. 2001. <https://agilemanifesto.org>
- [3] K. Beck. *Test-Driven Development: By Example*. Addison-Wesley, 2002.
- [4] GitHub. *Spec Kit: Toolkit for Spec-Driven Development*. 2025. <https://github.com/github/spec-kit>
- [5] E. Barlow. *Optimizing Specification-Driven Delivery: Atomic Decomposition, Build Graphs, and Context Engineering for Reproducible LLM Software Delivery*. Web Cloud Studio, 2026.